

Evals Report - Bhavya AI Persona (Yangoti Sai Bhavya)

1. Voice Quality

Built on Vapi with Deepgram (flux-general-en) for STT, GPT-4o as LLM, and Vapi's Naina voice for TTS. Evaluated across 8 manual test calls covering background Q&A, adversarial probing, and end-to-end booking.

Metric	Method	Result
First-response latency	Vapi call logs - bot speech start vs turn start	~1.15s average
Transcription accuracy	Manual review of Vapi transcripts vs spoken words	~94% - drops on proper nouns
Booking completion	Slot selected → name + email → Cal.com confirmed	8/8 successful

2. Chat Groundedness

RAG pipeline: resume split into 7 sections (section-aware chunking) + 13 GitHub READMEs at 120 words/chunk → all-MiniLM-L6-v2 embeddings → ChromaDB (109 chunks total). Evaluated by manually asking 15 questions drawn from resume and README content.

Metric	Method	Result
Hallucination rate	Manual check: did the answer contain anything not in the source?	0/15 (0%) - fully grounded
Retrieval quality	Did the right source chunk appear in retrieved context?	13/15 (87%) - 2 follow-up failures before fix

3. Three Failure Modes

F1: Vapi tool calls silently failing (Voice Agent)

The voice agent kept saying “I couldn’t retrieve availability” despite the backend being live. Vapi wraps function arguments in a nested structure (`message.toolCallList[0].function.arguments`), not a flat JSON body - causing FastAPI to return silent 422 errors and GPT-4o to hallucinate slots. Fixed by parsing the Vapi envelope via `async Request` and returning: `{"results":[{"toolCallId":"...","result":"..."}]}`.

F2: Follow-up questions losing conversation context (Chat)

Queries like “any other internships?” were retrieved in isolation, causing project chunks to rank above experience chunks. Fixed by prepending the previous user message to the current query before retrieval, carrying forward conversation topic across turns.

F3: Resume chunks losing to GitHub chunks in retrieval (RAG)

Fixed-window chunking split the Experience section into 4-5 weak fragments, which GitHub README chunks consistently outranked. Fixed by switching to section-aware chunking (one chunk per resume section) and force-including all resume sections in every retrieval context ahead of GitHub chunks.

4. Conscious Tradeoff: Token Cost vs. Factual Accuracy

A standard RAG system retrieves only the top-k most similar chunks. I chose to always inject all 7 resume sections into every query context, regardless of similarity score. This increases token usage by ~40% per query but completely eliminates the risk of GitHub data overriding verified resume facts. In a hiring tool where one hallucinated fact damages credibility, accuracy was prioritized over token efficiency. The overhead is acceptable at evaluation scale with Groq’s free tier.

5. What I’d Build With 2 More Weeks

- **Commit-level GitHub ingestion:** Index actual code files and commit history, not just READMEs, so the assistant can answer deep technical questions like “what design tradeoff did you make in the MALLM pipeline?”
- **Voice email fallback:** Offer a Cal.com booking link via SMS/chat when email transcription fails - eliminating the #1 voice booking failure mode entirely.
- **Automated eval CI pipeline:** Run a fixed golden Q&A set on every code push, tracking hallucination rate as a metric over time rather than relying on manual checks.